

Conception et développement d'IHM

Cours S4: Travailler avec les formulaires, les APIs, le routage

Introduction et prérequis: contenu

- Comment travailler avec les formulaires en React?
 - Les formulaires contrôlés
 - Les formulaires non contrôlés
 - La bibliothèque React Hook Form
- Effects
- Comment communiquer avec des APIs REST depuis React?
- Le routage
- Exemple d'une application CRUD (Create, Read, Update, Delete)

Comment travailler avec les formulaires en React?

Comment travailler avec les formulaires en React?

La création de formulaires dans React est un peu différente qu'en HTML et JavaScript pur. Il existe deux méthodes pour construire des formulaires dans React: les **formulaires contrôlés** et les **formulaires non contrôlés**.

- **Les formulaires contrôlés:** stockent la valeur de chaque entrée dans l'état React « state », puis définissent la valeur de chaque entrée à chaque nouveau rendu à partir de cet état. Si une autre fonction met à jour l'état, cela sera immédiatement reflété dans les valeurs des composants. Les entrées contrôlées ont tendance à donner plus d'options dans la logique React, comme la validation de formulaire complexe, lorsque l'utilisateur tape (par exemple en vérifiant la force du mot de passe), la manipulation du code des valeurs des entrées (par exemple en mettant un trait d'union entre les numéros de téléphone lorsque l'utilisateur tape).
- **Les formulaires non contrôlés:** ne sauvegardent pas les valeurs d'entrée dans l'état React « state ». Au lieu de cela, les formulaires non contrôlés utilisent les fonctionnalités <form> natives et intégrées de l'HTML et du JavaScript pour gérer les données. Ainsi, par exemple, au lieu de définir un état avec la valeur de l'entrée à chaque fois qu'elle est modifiée, puis de définir sa valeur, le navigateur gère la valeur de l'entrée. Au lieu d'utiliser l'état React, c'est beaucoup plus proche de la façon dont un simple formulaire HTML fonctionnerait sans aucun JavaScript.

Les formulaires contrôlés

```
import { useState } from "react";

function ControlledForm() {
  const [formData, setFormData] = useState({
    firstName: "",
    lastName: "",
    email: ""
  });

  const handleChange = e => {
    const { name, value } = e.target;
    setFormData(prevState => ({ ...prevState, [name]: value }));
  };

  return (
    <form>
      <label>First Name:</label>
      <input type="text" name="firstName" value={formData.firstName} onChange={handleChange} />
      <label>Last Name:</label>
      <input type="text" name="lastName" value={formData.lastName} onChange={handleChange} />
      <label>Email:</label>
      <input type="email" name="email" value={formData.email} onChange={handleChange} />
      <button onClick={handleSubmit}>Send</button>
    </form>
  );
}
```

Les inconvénients des formulaires contrôlés

- Il se peut que vous ne souhaitiez pas réafficher le formulaire à chaque fois que l'utilisateur saisit un texte.
- Vous devez écrire beaucoup de code pour gérer des formulaires complexes, parce que vous vous retrouvez avec beaucoup d'états au fur et à mesure que votre formulaire s'agrandit. Cela conduit à des `useState` et des `state-setting` répétitifs, qui alourdissent votre code.
- Il est plus difficile de construire des formulaires dynamiques où le nombre de champs est variable, parce que vous ne pouvez pas utiliser conditionnellement des hooks comme `useState`.
- Cette méthode peut entraîner des problèmes de performance avec des formulaires plus volumineux tels que les feuilles de calcul, les tableaux, etc.

Les formulaires non contrôlés

```
function UncontrolledForm() {
  const handleSubmit = e => {
    e.preventDefault();
    const formData = new FormData(e.target);
    for (let [key, value] of formData.entries()) {
      console.log(`${key}: ${value}`);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>First Name:</label>
      <input type="text" name="firstName" />
      <label>Last Name:</label>
      <input type="text" name="lastName" />
      <label>Email:</label>
      <input type="email" name="email" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

L'inconvénient des formulaires non contrôlés est que vous n'avez pas d'accès direct à la valeur de chaque champ. Cela peut rendre la validation personnalisée plus difficile et rendre impossible la manipulation directe de la valeur des champs.

Quelle est donc la meilleure méthode pour travailler avec les formulaires en React?

Chacune des deux méthodes: formulaires contrôlés et non contrôlés comporte des inconvénients. La meilleure méthode pour travailler avec les formulaires en React est de **mixer la méthode des formulaires contrôlés et non contrôlés**, afin de tirer profit des avantages offerts par chacune des deux méthodes tout en évitant les inconvénients. Nous pouvons accomplir ce mariage entre ces deux méthodes comme suite:

- Dans de nombreux cas, vous pouvez avoir besoin de contrôler un ou plusieurs champs. La manipulation d'un numéro de téléphone au fur et à mesure que l'utilisateur le tape en est un bon exemple. Dans ce cas, même si vous utilisez un formulaire non contrôlé, vous pouvez vous contenter d'un champ contrôlée. Dans ce cas, n'utilisez pas l'état pour accéder à la valeur du champ lors de la soumission du formulaire - continuez à utiliser `new FormData(...)` - utilisez simplement l'état pour gérer l'affichage du champ en question.
- Pour la validation, utiliser tout simplement la validation HTML5 native.
- L'un des avantages des formulaires contrôlés est la possibilité d'afficher des erreurs de validation côté client. Cependant, au lieu d'utiliser des formulaires contrôlés et de définir des valeurs à chaque changement d'entrée, lorsque les exigences le permettent, nous pouvons utiliser un formulaire non contrôlés et gérer la gestion des erreurs dans la fonction `onSubmit`. Cela minimise la quantité d'état et de mises à jour d'état dans les formulaires.

La meilleure méthode pour travailler avec les formulaires en React: exemple

```
function MixedForm() {
  const [phoneNumber, setPhoneNumber] = useState("");
  const [errors, setErrors] = useState({});
  const handlePhoneNumberChange = e => setPhoneNumber(e.target.value.replace(/\D/g, "").slice(0, 10));

  const handleSubmit = (event) => {
    event.preventDefault();
    const formData = new FormData(event.target);

    let validationErrors = {};
    if (formData.get("email") && !formData.get("email").endsWith("@example.com")) {
      validationErrors.email = "Email must be from the domain example.com.";
    }

    if (Object.keys(validationErrors).length > 0) { setErrors(validationErrors); } else {
      console.log(Array.from(formData.entries()));
      setErrors({});
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>Name:</label>
      <input type="text" name="name" />
      <label>Email:</label>
      <input type="email" name="email" />
      {errors.email && <div className="error">{errors.email}</div>}
      <label>Phone Number:</label>
      <input type="tel" name="phoneNumber" value={phoneNumber} onChange={handlePhoneNumberChange} />
      <button type="submit">Submit</button>
    </form>
  );
}
```

La bibliothèque React Hook Form

Même avec la méthode de couplage des deux méthodes de formulaires contrôlés et non contrôlés. Travailler avec les formulaires en React reste toujours un challenge. C'est pour cela que plusieurs bibliothèques ont été proposées au fur des années, pour faciliter la création des formulaires. La bibliothèque la plus utilisée actuellement (2024) est [React Hook Form](#). Cette bibliothèque réduit la quantité de code à écrire tout en supprimant les « re-renders » inutiles, et simplifie la validation et la gestion des erreurs. Ci-dessous, la commande d'installation avec un exemple basique d'utilisation.

```
npm install react-hook-form
```

```
import { useForm } from "react-hook-form"

export default function App() {
  const { register, handleSubmit, watch, formState: { errors } } = useForm()
  const onSubmit = data => console.log(data)
  console.log(watch("example"))
  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input defaultValue="test" {...register("example")} />
      <input {...register("exampleRequired", { required: true })} />
      {errors.exampleRequired && <span>This field is required</span>}
      <input type="submit" />
    </form>
  )
}
```

Effects

Effects

Certains composants doivent être synchronisés avec des systèmes externes. Par exemple, vous pourriez vouloir contrôler un composant non React en fonction de l'état React, établir une connexion avec un serveur ou envoyer un journal d'analyse lorsqu'un composant apparaît à l'écran. Les effets vous permettent d'exécuter du code après le rendu afin de **synchroniser votre composant avec un système extérieur à React**. « Effects » sont essentiellement utilisés pour faire **communiquer et synchroniser une interface React avec une API distante**.

Pour créer un « Effect », suivez les trois étapes suivantes:

1. **Déclarer un effet:** par défaut, votre effet sera exécuté après chaque rendu.
2. **Spécifiez les dépendances de l'effet:** la plupart des effets ne doivent être ré-exécutés qu'en cas de besoin, plutôt qu'après chaque rendu.
3. **Ajouter un nettoyage si nécessaire:** certains effets doivent préciser comment arrêter, annuler ou nettoyer ce qu'ils étaient en train de faire.

Créer un effet

Étape 1: déclarer un effet

Pour déclarer un effet dans votre composant, importez le hook **useEffect** de React. Ensuite, appelez-le au niveau supérieur de votre composant et mettez votre code à l'intérieur de votre Effect.

```
import { useEffect } from 'react';

function MyComponent() {
  useEffect(() => {
    // Le code ici sera exécuté après *chaque* rendu
  });
  return <div />;
}
```

Chaque fois que votre composant effectue un rendu, React met à jour l'écran et exécute ensuite le code contenu dans `useEffect`. En d'autres termes, `useEffect` "retarde" l'exécution d'un morceau de code jusqu'à ce que le rendu soit reflété à l'écran.

Étape 2 : Spécifier les dépendances de l'effet

Par défaut, les effets sont exécutés après chaque rendu. Si vous voulez que l'effet soit exécuter seulement lors du premier chargement du composant, passer un tableau de dépendances vide lors de la création de l'effet.

```
useEffect(() => {
  // Ce code ne s'exécute que lors du premier chargement
}, []);
```

Comment communiquer avec des APIs REST depuis React?

Comment communiquer avec des APIs REST depuis React?

Qu'est-ce qu'une API REST ?

API signifie Application Programming Interface (interface de programmation d'applications). Il s'agit d'un support qui permet à **différentes applications de communiquer entre elles de manière programmatique** et de renvoyer une réponse en temps réel.

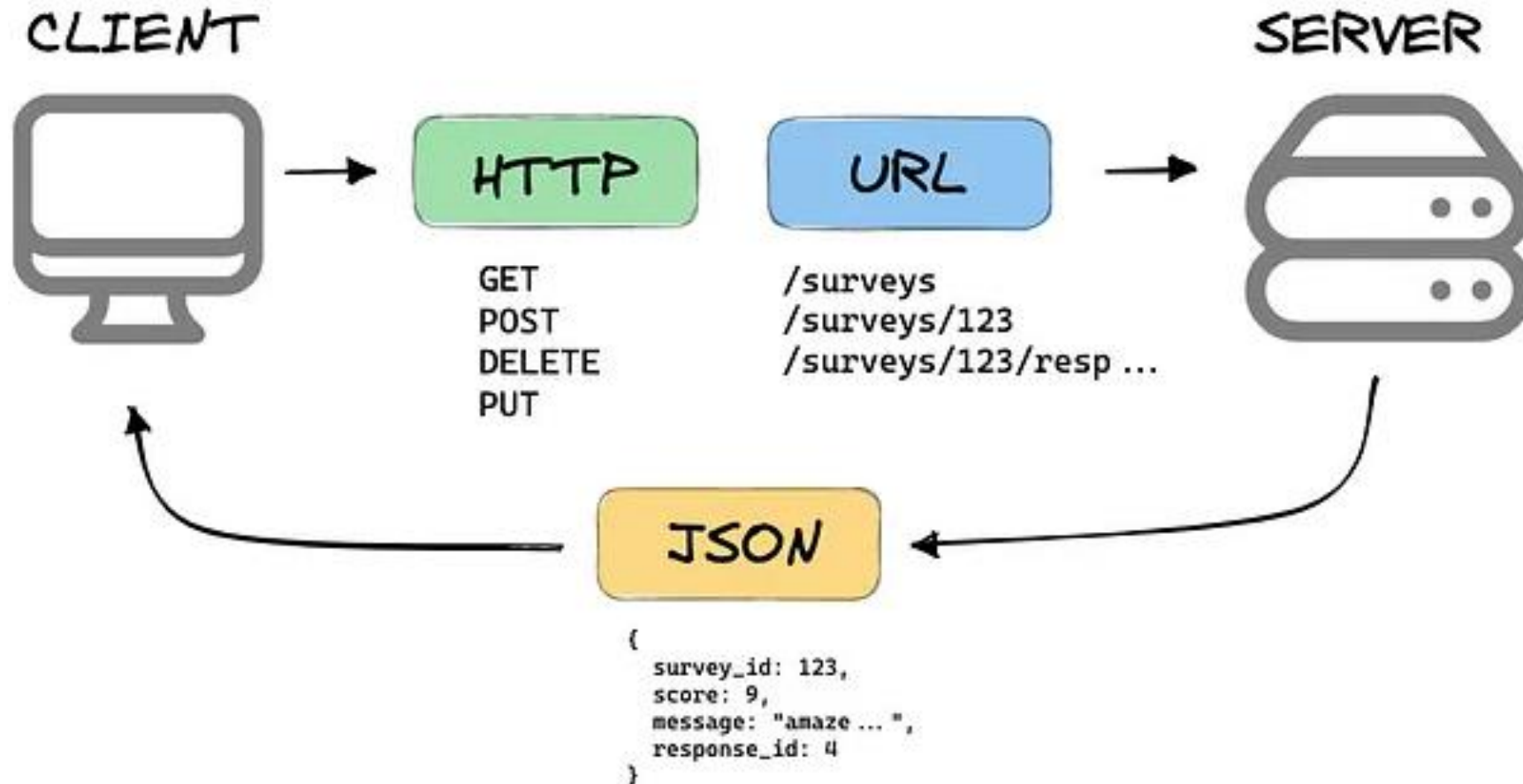
Roy Fielding a défini REST en 2000 comme un style architectural et une méthodologie couramment utilisés dans le développement de services Internet, tels que les systèmes hypermédias distribués. Il s'agit d'un acronyme qui signifie "REpresentational State Transfer" (transfert d'état représentationnel).

Tout développeur qui souhaite créer des applications web modernes et robustes avec React doit comprendre comment consommer des API pour récupérer des données dans les applications React.

Comment communiquer avec des APIs REST depuis React?

Echange client/API REST ?

Lorsqu'une requête est effectuée via une API REST, celle-ci envoie une représentation de l'état actuel de la ressource au demandeur ou au point de terminaison. Cette représentation de l'état peut prendre la forme de JSON, XML ou HTML. JSON est le format de fichier le plus largement utilisé car il est indépendant des langages de programmation et peut être lu à la fois par les humains et les machines.



Comment communiquer avec des APIs REST depuis React?

Comment consommer des API à l'aide de l'API Fetch?

L'API Fetch est une fonction JavaScript permettant de récupérer des ressources à partir d'un serveur ou d'une API. Elle est native, vous n'avez donc pas besoin d'installer de dépendances ou de paquets.

La méthode `fetch()` requiert un argument obligatoire, qui est le chemin ou l'URL de la ressource que vous souhaitez récupérer. Elle renvoie ensuite une promesse afin que vous puissiez gérer le succès ou l'échec à l'aide des méthodes `then()` et `catch()`.

Une requête fetch de base est très simple à écrire et ressemble au code ci-dessous. Nous récupérons simplement des données à partir d'une URL qui renvoie des données au format JSON, puis nous les enregistrons dans la console :

```
fetch('https://jsonplaceholder.typicode.com/posts?_limit=10')  
  .then(response => response.json())  
  .then(data => console.log(data));
```

La réponse par défaut est généralement une réponse HTTP normale plutôt que le JSON proprement dit, mais nous pouvons obtenir notre résultat sous la forme d'un objet JSON en utilisant la méthode `json()` de la réponse.

Une alternative de l'API Fetch est d'installer et utiliser la bibliothèque [Axios](#) qui simplifie d'avantage la consommation d'APIs.

React + API REST: comment effectuer une requête GET avec l'API Fetch

Pour illustrer comment nous pouvons communiquer avec des APIs REST depuis React. Nous allons travailler avec une API de test, gratuite, et disponible en ligne sur <https://jsonplaceholder.typicode.com>

```
import { useState, useEffect } from 'react';

const App = () => {
  const [posts, setPosts] = useState([]);
  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts?_limit=10')
      .then(response => response.json())
      .then(data => setPosts(data))
      .catch(err => console.log(err.message));
  }, []);

  return (
    <div className="posts-container">
      {posts.map((post) => {
        return (
          <div className="post-card" key={post.id}>
            <h2 className="post-title">{post.title}</h2>
            <p className="post-body">{post.body}</p>
            <div className="button">
              <div className="delete-btn">Delete</div>
            </div>
          </div>
        );
      })}
    </div>
  );
};
```

React + API REST: comment effectuer une requête GET avec l'API Fetch

Explications du code:

- Nous avons créé un état « state » pour stocker les données que nous récupérerons de l'API afin de pouvoir les afficher ultérieurement dans notre composant.
- Nous avons utilisé `useEffect` pour récupérer les données/postes dès le chargement de l'application. La requête de récupération produit une promesse, que nous pouvons consommer par la suite.
- Nous avons consommé la promesse en utilisant `.then()`, mais elle a renvoyé un objet `Response`, ce qui n'est pas ce que nous voulons. Nous devons donc convertir l'objet `Response` au format JSON à l'aide de la méthode `json()`. Cette méthode renvoie également une promesse qui nous permet d'obtenir les données réelles à l'aide de la deuxième méthode `.then()`.

```
const [posts, setPosts] = useState([]);
useEffect(() => {
  fetch('https://jsonplaceholder.typicode.com/posts?_limit=10')
    .then(response => response.json())
    .then(data => setPosts(data))
    .catch(err => console.log(err.message));
}, []);
```

React + API REST: comment effectuer une requête POST avec l'API Fetch

Vous pouvez utiliser la méthode HTTP POST pour envoyer des données à une API. Son fonctionnement est similaire à celui de la requête GET, la principale différence étant que vous devez ajouter la méthode et deux paramètres supplémentaires à l'objet optionnel.

```
const addPosts = async (title, body) => {
  await fetch('https://jsonplaceholder.typicode.com/posts', {
    method: 'POST',
    body: JSON.stringify({
      title: title,
      body: body,
      userId: Math.random().toString(36).slice(2),
    }),
    headers: { 'Content-type': 'application/json; charset=UTF-8' },
  })
  .then(response => response.json())
  .then(data => {
    setPosts(posts => [data, ...posts]);
    setTitle('');
    setBody('');
  })
  .catch(err => console.log(err.message));
};
```

Le corps contient les données que nous voulons transmettre à l'API. L'en-tête nous indique le type de données, qui est toujours le même lorsque l'on consomme des API REST. Nous resetons également l'état pour contenir les nouvelles données en plus des données existantes.

React + API REST: comment effectuer une requête POST avec l'API Fetch

En plus du code précédent. Nous avons besoin d'un formulaire pour pouvoir saisir les données.

```
import { useState } from 'react';
const App = () => {
  const [title, setTitle] = useState('');
  const [body, setBody] = useState('');
  const addPosts = async (title, body) => { /* voir slide précédent */ };
  const handleSubmit = (e) => {
    e.preventDefault();
    addPosts(title, body);
  };

  return (
    <div className="app">
      <div className="add-post-container">
        <form onSubmit={handleSubmit}>
          <input type="text" value={title} onChange={(e) => setTitle(e.target.value)} />
          <textarea value={body} onChange={(e) => setBody(e.target.value)}></textarea>
          <button type="submit">Add Post</button>
        </form>
      </div>
    </div>
  );
};

export default App;
```

React + API REST: comment effectuer une requête DELETE avec l'API Fetch

```
const deletePost = async (id) => {
  await fetch(`https://jsonplaceholder.typicode.com/posts/${id}`, {
    method: 'DELETE',
  }).then((response) => {
    if (response.status === 200) {
      setPosts(
        posts.filter((post) => {
          return post.id !== id;
        })
      );
    } else {
      return;
    }
  });
};
```

Ceci est déclenché lorsque le bouton de suppression est cliqué, et nous obtenons l'identifiant du post spécifique dans lequel le bouton a été cliqué. Nous supprimons ensuite ces données de l'ensemble des données renvoyées. Ces données seront supprimées de l'API mais pas immédiatement de l'interface utilisateur, c'est pourquoi nous avons ajouté un filtre pour supprimer également les données.

Le routage

Routage

Le routage est le processus de **redirection** d'un utilisateur vers différentes pages en fonction de son action ou de sa demande. Pour implémenter le routage dans React, on utilise la bibliothèque [React router](#). Cette bibliothèque vous permet de créer des applications web complexes et multipages qui ressemblent à des sites web traditionnels tout en étant des applications monopages (SPA). Avec React Router, vous pouvez définir et gérer des itinéraires, ce qui permet aux utilisateurs de naviguer dans différentes pages de votre application sans avoir à demander une nouvelle page HTML au serveur.

Les concepts clés de React Router:

- **Route:** Une route est une correspondance entre une URL et un composant. Lorsqu'un utilisateur visite une URL spécifique, React Router affiche le composant correspondant.
- **Router** (Le routeur) : Le routeur est le composant de premier niveau qui fournit l'infrastructure de routage. Dans React Router, on utilise généralement BrowserRouter.
- **Nested Routes** (Routes imbriquées) : React Router vous permet d'imbriquer les routes, en créant une hiérarchie de composants.
- **Link** (Lien) : Le composant Link permet la navigation en créant des éléments semblables à des ancres qui maintiennent l'état de l'application.

Routage

Etape 1: Installation de React Router:

Afin d'utiliser la bibliothèque React Router, il faut d'abord l'installer.

```
npm install react-router-dom
```

Etape 2:

Dans le fichier index.js, il faut mettre le composant initial App à l'intérieur du composant BrowserRouter.

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
import { BrowserRouter } from 'react-router-dom';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

Routage

Etape 3: Création de la barre de navigation:

Créer un fichier Navbar.js dans votre dossier src comme dans l'exemple ci-dessous.

```
import { Link } from "react-router-dom";

const Navbar = () => {
  return (
    <nav>
      <ul>
        <li>
          <Link to="/">Home</Link>
        </li>
        <li>
          <Link to="/about">About</Link>
        </li>
        <li>
          <Link to="/contact">Contact</Link>
        </li>
      </ul>
    </nav>
  );
}

export default Navbar
```

Routage

Etape 4: Création des pages

Dans l'exemple ci-dessous, nous allons créer trois pages qui correspondent aux trois route déjà définies.

```
//Home.js
import React from 'react'
import Navbar from './Navbar'

const Home = () => {
  return (
    <div>
      <Navbar />
      <h1>Home</h1>
    </div>
  )
}

export default Home
```

```
//About.js
import React from 'react'
import Navbar from './Navbar'

const About = () => {
  return (
    <div>
      <Navbar />
      <h1>About</h1>
    </div>
  )
}

export default About
```

```
//Contact.js
import React from 'react'
import Navbar from './Navbar'

const Contact = () => {
  return (
    <div>
      <Navbar />
      <h1>Contact</h1>
    </div>
  )
}

export default Contact
```

Routage

Etape 5: Création des routes dans App.js

```
//App.js
import { Route, Routes } from 'react-router-dom';
import './App.css';
import Home from './Home';
import About from './About';
import Contact from './Contact';

function App() {
  return (
    <div className="App">
      <Routes>
        <Route path="/" element={<Home/>} />
        <Route path="/about" element={<About/>} />
        <Route path="/contact" element={<Contact/>} />
      </Routes>
    </div>
  );
}

export default App;
```

- Les éléments « **Route** » sont enveloppés dans le composant « **Routes** ».
- « **path** » représente le chemin URL vers le composant qu'il représente.
- « **element** » décrit le composant vers lequel pointe le chemin

Routage dynamique

Dans React router, les paramètres URL sont des espaces réservés « placeholder » que vous déclarez dans une route, comme dans l'exemple ci-dessous.

```
<Routes>
  <Route path="/blog/:id" element={<Blog />} />
</Routes>
```

Dans l'exemple ci-dessus, « id » est un espace réservé en raison du « : » qui le précède. L'espace réservé est connu sous le nom de paramètre URL. À cause du paramètre URL, le routeur React ne fera pas littéralement correspondre la route ci-dessus avec l'URL. Il le fera **dynamiquement** si vous dirigez le navigateur vers une URL qui correspond au motif **/blog/:id**.

En supposant que vous hébergez le site sur <https://myreact.dev/>, pointer le navigateur vers <https://myreact.dev/blog/3> correspondra à la route ci-dessus. Par conséquent, le paramètre id prendra la valeur "3" dans le composant Blog.

Routage dynamique

Supposant que vous ayez la route dans le slide précédent dans votre configuration de routage, vous pouvez récupérer les paramètres de la route dans le composant Blog en utilisant le « hook » **useParams**. Il renvoie un objet. Les clés de l'objet sont les noms des paramètres déclarés dans la chaîne de chemin dans la définition de la route, et les valeurs sont les segments d'URL correspondants de l'URL correspondante.

```
import { useParams } from "react-router-dom";

const Blog = () => {
  const routeParams = useParams();
};
```

Si la route correspondante est **/blog/3** dans l'exemple ci-dessus, le « hook » **useParams** renverra l'objet ci-dessous.

```
{ id: "3" }
```

Exemple d'une application CRUD (voir démo)
